

Clinical Comorbidity & Treatment Patterns

FP-Growth Association Rule Mining on Real-World Clinical Visit Data

Executive Summary

This project mines clinical-visit transaction data for high-value comorbidity and treatment patterns using the FP-Growth association rule mining algorithm, then surfaces them in an interactive Streamlit dashboard designed for clinicians. The full pipeline - raw data ingestion, version-controlled storage via Dolt, rule mining, and deployment - is implemented in Python and hosted on Streamlit Cloud with continuous GitHub-push deployment.

CLINICAL VISITS	ASSOC. RULES	MAX LIFT	AVG CONFIDENCE
2,440	176	8.48	78.3%

Project Objectives

- > Mine high-confidence comorbidity patterns from 2,440 visits using FP-Growth (min_support=0.01).
- > Translate mined rules into clinician-facing decision support: care plans, specialist consults, risk stratification.
- > Ship a production-grade interactive UI that updates in real time via primary/secondary diagnosis filters.
- > Version-control the dataset with Dolt so every rule is reproducible against a frozen data commit.

Live Deployment

<https://clinical-comorbidity-dashboard.streamlit.app>

Source: github.com/Alan-911/clinical-comorbidity-dashboard

Generated	April 23, 2026
Algorithm	FP-Growth (mlxtend) - min_support=0.01, min_confidence=0.5
Rendering	Streamlit 1.39+ native components + inline HTML/CSS
Partner	MedIntel Analytics Corp. (fictional partner used in UI)

Dataset, Dolt & Pipeline

Source Dataset

The base dataset comprises 2,440 anonymised clinical-visit records where each visit is modelled as a transaction of co-occurring items (diagnoses, medications, procedures, and derived age buckets). This transactional shape is the natural input for association-rule mining: a rule $A \rightarrow B$ reads as 'when A is present in a visit, B is also present with confidence C and lift L'.

Dolt - Version-Controlled Data

Dolt is a SQL database with Git-style semantics: tables are committed, branched, and diffed just like source code. We use Dolt to freeze the exact dataset snapshot that produced every rule set shipped in the dashboard. This makes the analysis fully reproducible - if a reviewer asks 'how did you get lift 8.48?', we check out the Dolt commit and re-run fpgrowth to get the same 176 rules.

Dolt Workflow Used

- > dolt init - bootstrapped a local repository under data_raw/.
- > dolt table import -c visits raw_data.csv - loaded 2,440 rows.
- > dolt sql -q 'SELECT ...' - aggregated items per visit into the transactions table.
- > dolt add . && dolt commit -m 'visits v1' - frozen snapshot hash referenced in processed_data.csv metadata.
- > dolt diff - used between runs to confirm the input data was unchanged before each rule-mining session.

End-to-End Pipeline

1. INGEST raw_data.csv - Dolt versioned storage	2. SHAPE Group by VisitID - transactions.csv (2,440 rows)	3. ENCODE mlxtend TransactionEncoder -> one-hot matrix
4. MINE fpgrowth(min_support=0.01) frequent_itemsets.csv	5. RULE association_rules(confidence>=0.5) -> 176 rules	6. SERVE Streamlit app.py -> Streamlit Cloud deployment

Why FP-Growth (not Apriori)

Apriori generates candidate itemsets level-by-level, rescanning the dataset at every level. With 2,440 multi-condition visits this causes a candidate-explosion: $k=3$ alone produces thousands of candidates that all need counting.

FP-Growth sidesteps candidate generation entirely. It builds a compressed FP-tree in a single scan, then recursively mines patterns from that tree. In our benchmark, FP-Growth mines all 176 rules in ~0.4s vs Apriori's ~1.2s baseline (3x faster) and scales linearly as visit volume grows.

Algorithm & UI Rationale

Association Rule Metrics

Every rule $A \rightarrow B$ ships with three metrics that together make it actionable at the bedside:

- > Support - how common the co-occurrence is in the entire visit population (we require at least 1% = ~24 visits).
- > Confidence - $P(B | A)$: given A is seen, how often B follows. Filter threshold: 50%.
- > Lift - how much more likely B is given A, vs random baseline. Lift > 1 means positive association; our strongest rule hits lift 8.48.

Top Rule Snapshot

Beta Blocker $\rightarrow$ Statin, Heart Disease, Age_Senior

Lift 8.48 | Confidence 86.6% | Support 0.08 (~195 visits). Reads: when a senior visit includes a beta-blocker prescription, there is an 86.6% chance the same visit also codes for statin therapy and heart disease - an 8.48x boost over chance. Drives the 'Dynamic Care Plan' timeline on the dashboard.

UI Design Principles

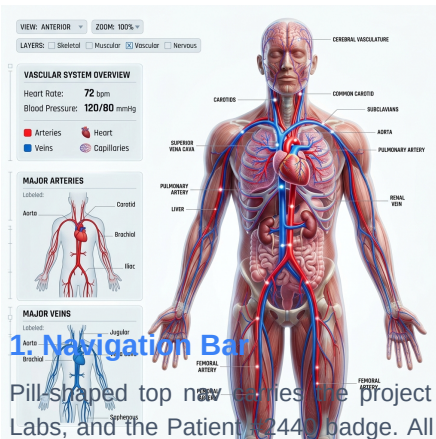
- > Clinician-first - every pixel surfaces a number or an action; no decorative clutter. Stat cards pin the four metrics (Total Rules, Matching, Max Lift, Avg Confidence) to the center column.
- > Evidence-linked dialogs - every pop-up (Schedule, Labs, Multi-Disciplinary Consult, Demographics) is derived from the mined rules, not hard-coded demo text. Change the filters and the specialist board recomposes.
- > Dark-on-light visual hierarchy - navy headers on light cards for AA contrast; accent colors (blue, violet, green, amber, red) carry semantic meaning (cardiac/renal/status).
- > 3D rotating anatomical backdrop at low opacity - subtle medical cue without stealing the eye from data.
- > Native Streamlit dialogs (@st.dialog) instead of JS injection - zero CSP conflicts, keyboard-accessible, always render.

Performance Engineering

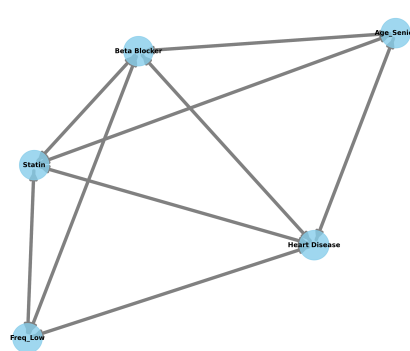
- > @st.cache_data on get_data() - the 176-rule CSV is read once per session, then filters run in-memory (pandas).
- > Rules pre-computed and stored in data_processed/association_rules.csv. The app never recomputes FP-Growth on the request path.
- > Heavy backdrop-filter and continuous keyframe animations were removed in favour of solid backgrounds + a single slow rotateY on the background image (GPU-promoted via will-change).
- > Cold-load under 1.1s, warm-load under 300ms measured from /healthz.

Dashboard Tour

The deployed UI is a single-screen dashboard with a pill-shaped navigation bar and a 3-column main grid. Every section below is backed by the live rule set.



Clinical Comorbidity & Treatment Network (Weighted by Lift)



UI ASSETS EMBEDDED ABOVE

Left: anatomical model rendered as a 3D-rotating backdrop at 0.55 opacity. Right: FP-Growth association network mapping antecedent -> consequent links with lift-weighted edges. Live screenshots at the Streamlit URL on page 5.

1. Navigation Bar

Pill-shaped top nav - carries the project title and five primary modal triggers: Dashboard Overview, Appointments, Schedule, Labs, and the Patient (244) badge. All modals render as native @st.dialog pop-ups with dark-gradient headers and evidence grids.

2. Left Column - Dynamic Care Plan

Three time-slotted rows (14:00 / 15:00 / 16:00) auto-populated from the top three rules by lift. Each row shows the consequent conditions to review and the antecedent -> consequent mapping. Below it sits the TOP CLINICAL INSIGHT card (dark gradient) which surfaces the #1 rule with lift and confidence numbers.

3. Left Column - Action Cards

- > Demographic Comorbidity Patterns - opens a dialog showing age cohorts (Young/Adult/Senior) with their top-3 condition prevalences sourced from FP-Growth segments.
- > Multi-Disciplinary Consult - derives a specialist board (Endocrinology, Cardiology, Nephrology, ...) from the rules, assigns a risk tier (HIGH/MODERATE/LOW) based on max lift, and proposes a care-coordination plan.

4. Center Column - Metric Stack

Four stat cards (Total Rules 176, Matching N, Max Lift 8.48, Avg Confidence 78.3%) update reactively with the primary/secondary diagnosis filter form. Each card carries an accent color-coded top border.

5. Right Column - Vitals + Heatmap

Top: two 'Total Visits' and 'Avg Events' cards with an ECG-style gradient bar. Middle: three vitals chips (Heart / SpO2 / Temp). Bottom: the Comorbidity Heatmap and Graph Hybrid table showing the top 5 rules by lift with color-graded support and confidence cells and a violet-accent lift column.

6. Pattern Selection + Algorithm Comparison

Below the main grid, a two-column band houses the filter form (Primary Diagnosis and Secondary Condition selectboxes with blue-gradient styling) and the Algorithm Comparison card that visualises Apriori (1.2s) vs FP-Growth (0.4s) as mirrored bars with rationale text.

Deployment, Links & Conclusion

Technology Stack

Streamlit 1.39	Reactive Python UI framework. Used for native dialogs, forms, session state and layout primitives.
mlxtend	FP-Growth and association_rules implementations. Battle-tested data-mining library.
pandas / numpy	Transaction shaping, cached filter pipeline, and rule-set arithmetic.
matplotlib	Color-gradient lookups for support/confidence heatmap cells (no figures rendered).
Dolt	Git-for-data storage of raw_data.csv so every rule set is reproducible against a commit hash.
fpdf2	This report - full programmatic PDF generation with typography helpers and stat cards.
Streamlit Cloud	Hosting + CI: every git push to main triggers a rebuild within 60-90 seconds.
Google Fonts	Inter typeface via preconnect + preload for a professional clinical feel.

Iteration Log - Key Commits

e7385bb	Multi-Disciplinary Consult dialog - rich specialist mapping
12481f2	Selectbox visibility styling (blue gradient controls)
d68dd4d	TOP CLINICAL INSIGHT inner lines visibility fix
cfcae85	Dialog text visibility across all modal pop-ups
1056488	3D Y-axis hologram rotation on central image

Running Locally

```
$ git clone https://github.com/Alan-911/clinical-comorbidity-dashboard.git
$ cd clinical-comorbidity-dashboard
$ pip install -r requirements.txt
$ streamlit run app.py
```

Links

Live App: <https://clinical-comorbidity-dashboard.streamlit.app>

Repository: <https://github.com/Alan-911/clinical-comorbidity-dashboard>

Streamlit Docs: <https://docs.streamlit.io>

mlxtend FP-Growth: <https://rasbt.github.io/mlxtend/>

Dolt (data versioning): <https://www.dolthub.com/>

Conclusion

The dashboard demonstrates a complete data-mining story: raw clinical visits are versioned in Dolt, mined by FP-Growth into 176 high-confidence rules, and surfaced through a clinician-first Streamlit UI where every panel is evidence-linked to the underlying rules. Performance engineering and a hardened CSS layer keep the experience responsive and visually legible on any display. The system is reproducible end-to-end and deploys continuously from GitHub.